System Design Document — TaskAnalyzer

Especificação técnica executável (SDD) — versão 1.1.0

Repositório: github.com/PedroVargas1204/sdd-task-analyzer
Documento gerado automaticamente por `tools/gerar_pdfs.py` a partir de `specs/task_analyzer_spec.md`
Data de geração: 25/10/2026

Histórico de versões

Versão	Data	Autor	Descrição
1.0.0	13/09/2026	Pedro Vargas dos Santos e Silva	Versão inicial (Fase 1): contrato, cenários, CONTEXT_RULES e arquitetura.
1.1.0	22/09/2026	Pedro Vargas dos Santos e Silva	Alinhamento ao enunciado da Fase 2 (ver "Mudanças em relação à v1.0.0").

Mudanças em relação à v1.0.0

O enunciado da Fase 2 trouxe requisitos novos. Eles foram tratados como **mudança de requisito do cliente**: o contrato foi revisado e versionado **antes** da geração de código, para que a IA nunca precise escolher entre duas fontes conflitantes.

#	v1.0.0	v1.1.0	Motivo
M1	`analisar_tarefas`	`analyze_tasks`	Nome exigido pelo enunciado da Fase 2.
M2	Base `TaskAnalyzerError`	Base `TaskValidationError` (subclasses mantidas)	Exceção exigida pelo enunciado; a precisão das subclasses é preservada.
M3	Lista vazia → `EntradaInvalidaError`	Lista vazia → relatório com zeros e 0.0	Caso de borda exigido: "retorno de 0.0 sem exceções".
M4	Sem concluídas → `SemTarefasConcluidasError`	Sem concluídas → médias 0.0 (exceção removida)	Idem; a divisão por zero continua impossível (guarda explícita).
M5	Tempo em minutos (`_min`)	Tempo em horas (`_horas`)	Métrica `tempo_medio_conclusao_horas` exigida.
M6	`quantidade_*` e `quantidade_por_status`	`total_tarefas`, `total_concluidas`, `total_pendentes`, `total_canceladas`, `total_atrasadas`	Nomes exigidos pelo enunciado.
M7	Prioridade sem concluídas → `None`	Prioridade sem concluídas → 0.0	Coerência com M3/M4 e tipos mais simples (`float`).
M8	Branches `main`/`develop`/`feature`	Branches `main`/`feature`, PR direto para `main`	Fluxo exigido pelo enunciado.

---

1. Visão Geral e Contrato de Negócio

1.1 Propósito

O **TaskAnalyzer** recebe uma lista de tarefas já registradas e produz um relatório de métricas de produtividade que responde a três perguntas:

1. **Quanto tempo levamos para concluir uma tarefa?** Tempo médio de conclusão, em horas, geral e por prioridade.
2. **Com que frequência entregamos atrasado?** Taxa de atraso percentual, geral e por prioridade.
3. **Como está a distribuição do trabalho?** Totais por status e por prioridade.

Dentro do escopo: validação rigorosa da entrada; cálculo de tempo médio e taxa de atraso (gerais e por prioridade); contagens; erros tratados por exceções específicas.

Fora do escopo: persistência, interface gráfica, API REST, CLI; leitura de JSON/CSV ou conversão de strings em datas; análise de backlog vencido (pendentes); horas úteis — o tempo é sempre **corrido**.

1.2 Requisitos não funcionais

ID	Requisito	Critério verificável
RNF01	Linguagem	Python 3.11 ou superior.
RNF02	Dependências	Apenas biblioteca padrão em `src/`; `pytest` e `pytest-cov` nos testes.
RNF03	Desempenho	O(n) no número de tarefas; sem laços aninhados sobre a lista.
RNF04	Efeitos colaterais	Função pura: sem I/O, sem estado global, entrada não modificada.
RNF05	Observabilidade	Logs via `logging`; o campo `titulo` nunca é registrado.
RNF06	Qualidade	Cobertura ≥ 90% em `src/` e 100% dos cenários aprovados.

1.3 Contrato executável de interface

Todos os nomes abaixo são **obrigatórios e literais**.

1.3.1 Entradas

A função principal recebe `tarefas: list[Tarefa]`. `Tarefa` é uma dataclass imutável:

Campo	Tipo	Obrigatório	Restrições	
`id_tarefa`	`int`	Sim	Inteiro > 0, único na lista. `bool` não é aceito.	
`titulo`	`str`	Sim	Não vazio nem só espaços. Nunca registrado em logs.	
`prioridade`	`Prioridade` (StrEnum)	Sim	`"baixa"`, `"media"`, `"alta"` (case-sensitive).	
`status`	`Status` (StrEnum)	Sim	`"concluida"`, `"pendente"`, `"cancelada"`.	
`data_criacao`	`datetime`	Sim	Timezone-aware.	
`prazo`	`datetime`	Sim	Timezone-aware; ≥ `data_criacao`.	
`data_inicio`	`datetime | None`	Condicional	Obrigatória se `concluida`; se informada, ≥ `data_criacao`.	
`data_conclusao`	`datetime | None`	Condicional	Obrigatória se `concluida`; `None` para `pendente`/`cancelada`; ≥ `data_criacao` e ≥ `data_inicio`.	

1.3.2 Saídas

`analyze_tasks` retorna `RelatorioMetricas` (dataclass imutável):

Campo	Tipo	Descrição
`total_tarefas`	`int`	Total de tarefas recebidas.
`total_concluidas`	`int`	Tarefas com status `concluida`.
`total_pendentes`	`int`	Tarefas com status `pendente`.
`total_canceladas`	`int`	Tarefas com status `cancelada`.
`total_atrasadas`	`int`	Concluídas após o prazo.
`tempo_medio_conclusao_horas`	`float`	Média geral em horas, 2 casas; 0.0 sem concluídas.
`taxa_atraso_percentual`	`float`	Percentual (0 a 100), 2 casas; 0.0 sem concluídas.
`indicadores_por_prioridade`	`dict[str, MetricasPrioridade]`	Chaves sempre presentes: `baixa`, `media`, `alta`.

`MetricasPrioridade` (dataclass imutável): `total_tarefas: int`, `total_concluidas: int`, `total_atrasadas: int`, `tempo_medio_conclusao_horas: float`, `taxa_atraso_percentual: float` — com as mesmas regras de cálculo, restritas à prioridade.

1.3.3 Regras de negócio

ID	Regra
RN01	Somente tarefas `concluida` entram no tempo médio e na taxa de atraso. Todas as tarefas válidas entram nas contagens.
RN02	Tempo de conclusão = `(data_conclusao − data_inicio).total_seconds() / 3600`, em horas (tempo corrido).
RN03	Uma concluída está atrasada se, e somente se, `data_conclusao > prazo`. Concluir exatamente no prazo não é atraso.
RN04	Tempo médio = soma dos tempos ÷ quantidade de concluídas do grupo.
RN05	Taxa de atraso = (atrasadas ÷ concluídas) × 100 do grupo.
RN06	`round(valor, 2)` aplicado **somente** ao valor final (arredondamento padrão do Python).
RN07	Todas as datas devem ser timezone-aware e são normalizadas para UTC antes de comparar ou calcular. Data naive ou de tipo diferente de `datetime` → `DataInvalidaError`.
RN08	Coerência cronológica: `prazo ≥ data_criacao`; `data_conclusao ≥ data_criacao`; `data_inicio ≥ data_criacao`; `data_conclusao ≥ data_inicio`. Violação → `DataInvalidaError`.
RN09	`concluida` exige `data_inicio` e `data_conclusao`; `pendente` e `cancelada` exigem `data_conclusao = None`. Violação → `TarefaInvalidaError`.
RN10	**Divisão por zero:** se o grupo não tiver concluídas, tempo médio e taxa valem **0.0**; a divisão nunca é executada e nenhuma exceção é lançada.
RN11	As três chaves de prioridade sempre existem; prioridade sem tarefas tem contagens 0 e métricas 0.0.
RN12	`prioridade` e `status` aceitam apenas os valores de 1.3.1. Outro valor → `TarefaInvalidaError`.
RN13	`id_tarefa` inteiro > 0 (bool proibido) e `titulo` não vazio → senão `TarefaInvalidaError`; ID duplicado → `EntradaInvalidaError`.
RN14	A entrada deve ser `list` (pode ser vazia) cujos elementos são todos `Tarefa`. Senão → `EntradaInvalidaError`. Lista vazia retorna relatório com zeros.
RN15	Validação fail-fast e atômica: todas as tarefas são validadas, na ordem da lista, antes de qualquer cálculo; nenhum resultado parcial é retornado.
RN16	Pureza: não modifica a entrada, não faz I/O e o resultado independe da ordem das tarefas.

Decisão registrada: tarefas com datas inconsistentes são **rejeitadas com exceção** (e não ignoradas), pois ignorá-las distorceria as métricas sem que o usuário percebesse.

Nota sobre 0.0: quando não há concluídas, 0.0 significa "sem dados", não "tempo zero". O chamador distingue os casos por `total_concluidas == 0`.

Ordem de validação por tarefa: (1) `id_tarefa` → (2) `titulo` → (3) `prioridade` → (4) `status` → (5) tipo e fuso das datas → (6) status × datas → (7) cronologia, na ordem de RN08. Após validar todas as tarefas, verifica-se a unicidade dos IDs.

1.3.4 Exceções

Todas herdam de `TaskValidationError(Exception)`, permitindo capturar qualquer erro do módulo com um único `except TaskValidationError`.

Exceção	Quando	Exemplo de mensagem
`EntradaInvalidaError`	Entrada não é `list`; elemento não é `Tarefa`; ID duplicado.	`"id_tarefa duplicado: 3."`
`TarefaInvalidaError`	ID inválido, título vazio, domínio inválido, status × datas.	`"Tarefa 5: valor inválido para prioridade: 'urgente'."`
`DataInvalidaError`	Tipo de data errado, data sem fuso, cronologia inválida.	`"Tarefa 7: data_conclusao anterior a data_criacao."`

Toda mensagem referente a uma tarefa começa com `Tarefa {id_tarefa}:`.

1.3.5 Assinaturas públicas (imutáveis)

```
class Prioridade(StrEnum): BAIXA = "baixa"; MEDIA = "media"; ALTA = "alta"
class Status(StrEnum): CONCLUIDA = "concluida"; PENDENTE = "pendente"; CANCELADA = "cancelada"

@dataclass(frozen=True)
class Tarefa:
    id_tarefa: int
    titulo: str
    prioridade: Prioridade
    status: Status
    data_criacao: datetime
    prazo: datetime
    data_inicio: datetime | None = None
    data_conclusao: datetime | None = None

@dataclass(frozen=True)
class MetricasPrioridade:
    total_tarefas: int
    total_concluidas: int
    total_atrasadas: int
    tempo_medio_conclusao_horas: float
    taxa_atraso_percentual: float

@dataclass(frozen=True)
class RelatorioMetricas:
    total_tarefas: int
    total_concluidas: int
    total_pendentes: int
    total_canceladas: int
    total_atrasadas: int
    tempo_medio_conclusao_horas: float
    taxa_atraso_percentual: float
    indicadores_por_prioridade: dict[str, MetricasPrioridade]

class TaskValidationError(Exception): ...
class EntradaInvalidaError(TaskValidationError): ...
class TarefaInvalidaError(TaskValidationError): ...
class DataInvalidaError(TaskValidationError): ...

def analyze_tasks(tarefas: list[Tarefa]) -> RelatorioMetricas: ...
def validar_tarefa(tarefa: Tarefa) -> None: ...
def calcular_tempo_conclusao_horas(tarefa: Tarefa) -> float: ...
def esta_atrasada(tarefa: Tarefa) -> bool: ...
```

Função	Responsabilidade	Exceções
`analyze_tasks`	Orquestra: valida a entrada e cada tarefa, calcula e monta o relatório.	Toda a hierarquia
`validar_tarefa`	Valida uma tarefa isolada (RN07–RN09, RN12, RN13).	`TarefaInvalidaError`, `DataInvalidaError`
`calcular_tempo_conclusao_horas`	Tempo de conclusão sem arredondamento (RN02).	`TarefaInvalidaError` se não concluída
`esta_atrasada`	`True` se `data_conclusao > prazo` (RN03).	`TarefaInvalidaError` se não concluída

---

2. Cenários de Aceite e Test Harness

2.1 Convenções

- Datas de março de 2026, em UTC salvo indicação; `data_criacao = 01/03/2026 08:00 UTC`.
- `CA-xx` = sucesso/borda; `CE-xx` = exceção/erro. O "Quando" é sempre `analyze_tasks(lista)` for executado (exceto CE-16).
- Valores esperados já arredondados (RN06) e comparados com igualdade exata.

2.2 Cenário 1 (Sucesso) — conjunto base (`tarefas_cenario_base`)

ID	Prioridade	Status	Início	Conclusão	Prazo	Tempo (h)	Atrasada?
1	alta	concluida	02/03 09:00	02/03 11:00	02/03 12:00	2.0	Não
2	alta	concluida	02/03 13:00	02/03 17:00	02/03 16:00	4.0	Sim
3	media	concluida	03/03 09:00	03/03 10:30	03/03 12:00	1.5	Não
4	media	pendente	—	—	05/03 18:00	—	n/a
5	baixa	concluida	04/03 08:00	04/03 08:45	04/03 08:45	0.75	Não (no limite)
6	baixa	cancelada	—	—	06/03 18:00	—	n/a

ID	Dado	Então
CA-01	Conjunto base	`total_tarefas == 6`, `total_concluidas == 4`, `total_pendentes == 1`, `total_canceladas == 1`, `total_atrasadas == 1`, `tempo_medio_conclusao_horas == 2.06` (8.25 ÷ 4 = 2.0625), `taxa_atraso_percentual == 25.0`.
CA-02	Conjunto base	alta: 2 / 2 / 1, 3.0 h, 50.0%; media: 2 / 1 / 0, 1.5 h, 0.0%; baixa: 2 / 1 / 0, 0.75 h, 0.0% (total / concluídas / atrasadas).

2.3 Cenário 2 (Exceção/Erro) — datas inconsistentes

ID	Dado	Então
CE-01	Conjunto base + tarefa 7 com `data_criacao = 02/03 12:00` e `data_conclusao = 02/03 10:00` (conclusão anterior à criação).	`TaskValidationError` (concretamente `DataInvalidaError`) com mensagem `"Tarefa 7: data_conclusao anterior a data_criacao."`; nenhum relatório parcial (RN08, RN15).
CE-02	Tarefa 7 com início 02/03 09:00 e conclusão 02/03 08:00.	`DataInvalidaError` com `"Tarefa 7: data_conclusao anterior a data_inicio"`.

2.4 Casos de borda e complementares de sucesso

ID	Dado	Então
CA-03	Alta concluída 09:00→10:00 com prazo 10:00.	0 atrasadas, taxa 0.0, tempo 1.0 (RN03).
CA-04	Três alta concluídas de 1 h, 1 h e 2 h; só a de 2 h atrasada.	Tempo 1.33 (4 ÷ 3) e taxa 33.33 (RN06).
CA-05	Alta concluída + media pendente; nenhuma baixa.	media: 1 tarefa, 0 concluídas, 0.0 / 0.0; baixa: tudo 0 e 0.0; três chaves (RN11).
CA-06	Base + tarefa 8 media pendente com prazo 01/03 12:00 (vencido).	Tempo 2.06 e taxa 25.0 inalterados; 7 tarefas; 2 pendentes (RN01).
CA-07	Base, cópia profunda e lista invertida.	Entrada inalterada; relatórios iguais (RN16).
CA-08	Início 02/03 06:00-03:00 (09:00 UTC), conclusão 10:00 UTC, prazo 09:30-03:00.	Tempo 1.0 e taxa 0.0 (RN07).
CA-09	Lista vazia `[]`.	Todos os totais 0; tempo e taxa 0.0 geral e por prioridade; sem exceção (RN10, RN14).
CA-10	Apenas tarefas pendente e cancelada.	Totais corretos; tempo e taxa 0.0; sem exceção (RN10).
CA-11	Tarefa com título `"DADO-SIGILOSO-123"`, analisada com sucesso e com erro, logs em DEBUG.	O título não aparece em nenhum log (RNF05).

2.5 Complementares de exceção

ID	Dado	Exceção esperada	Regra
CE-03	Entrada `None`, `dict`, `str` ou `tuple`.	`EntradaInvalidaError`	RN14
CE-04	Lista com um elemento que não é `Tarefa`.	`EntradaInvalidaError`	RN14
CE-05	Tarefa 5 com prioridade `"urgente"`.	`TarefaInvalidaError` "Tarefa 5"	RN12
CE-06	Tarefa 5 com status `"feito"`.	`TarefaInvalidaError` "Tarefa 5"	RN12
CE-07	Tarefa 5 com `data_inicio` naive.	`DataInvalidaError` "Tarefa 5"	RN07
CE-08	Tarefa 5 concluída sem `data_conclusao` ou sem `data_inicio`.	`TarefaInvalidaError` "Tarefa 5"	RN09
CE-09	Tarefa 5 pendente ou cancelada com `data_conclusao`.	`TarefaInvalidaError` "Tarefa 5"	RN09
CE-10	Duas tarefas com `id_tarefa = 3`.	`EntradaInvalidaError` "id_tarefa duplicado: 3"	RN13
CE-11	`id_tarefa` igual a 0, -1, `True` ou `"1"`.	`TarefaInvalidaError`	RN13
CE-12	Tarefa 5 com `data_inicio` anterior a `data_criacao`.	`DataInvalidaError` "Tarefa 5"	RN08
CE-13	Tarefa 5 com `prazo` anterior a `data_criacao`.	`DataInvalidaError` "Tarefa 5"	RN08
CE-14	Tarefa 5 com `titulo = " "`.	`TarefaInvalidaError` "Tarefa 5"	RN13
CE-15	Tarefa 5 com data em `str` ou `prazo = None`.	`DataInvalidaError` "Tarefa 5"	RN07
CE-16	`calcular_tempo_conclusao_horas` / `esta_atrasada` com tarefa pendente.	`TarefaInvalidaError` "Tarefa 5"	RN02, RN03

2.6 Planejamento e critérios do Test Harness

Fluxo: **cenários escritos** → **testes pytest (**`tests/test_harness.py`**)** → **validação do código gerado por IA**. Os testes são escritos e executados **antes** do código (fase vermelha) e depois com o código (fase verde).

Técnicas: fixtures `tarefas_cenario_base` e `tarefa_valida`; helper `utc()`; `dataclasses.replace` para variações; `@pytest.mark.parametrize` para CE-03 a CE-16; `pytest.raises(..., match=...)`; `caplog` para CA-11.

Critérios de aprovação: 100% dos 27 cenários (11 CA + 16 CE) aprovados; cobertura ≥ 90% em `src/`; nenhum `skip`/`xfail`; nenhum teste alterado pela IA (verificado no diff do PR).

Comando: `python -m pytest -v --cov=src --cov-report=term-missing`

2.7 Matriz de rastreabilidade

Regra	Cenários	Regra	Cenários
RN01	CA-01, CA-06	RN09	CE-08, CE-09
RN02	CA-01, CA-08, CE-16	RN10	CA-09, CA-10
RN03	CA-01, CA-03, CE-16	RN11	CA-05
RN04	CA-01, CA-02	RN12	CE-05, CE-06
RN05	CA-01, CA-02	RN13	CE-10, CE-11, CE-14
RN06	CA-04	RN14	CA-09, CE-03, CE-04
RN07	CA-08, CE-07, CE-15	RN15	CE-01
RN08	CE-01, CE-02, CE-12, CE-13	RN16	CA-07
RNF05	CA-11		

---

3. Governança de Contexto

As regras para agentes de IA estão em `CONTEXT_RULES.md`. Precedência em caso de conflito: (1) esta especificação → (2) `tests/test_harness.py` → (3) `CONTEXT_RULES.md` → (4) instruções pontuais na conversa.

---

4. Arquitetura do Repositório e Versionamento

```
sdd-task-analyzer/
■■■ README.md                  # Visão geral, execução e status
■■■ CONTEXT_RULES.md           # Regras persistentes para a IA
■■■ .gitignore                 # Arquivos ignorados
■■■ requirements.txt           # pytest e pytest-cov (versões fixadas)
■■■ specs/
■   ■■■ task_analyzer_spec.md  # Esta especificação
■■■ tests/
■   ■■■ test_harness.py        # Test Harness (pytest)
■■■ src/
    ■■■ task_analyzer.py       # Código gerado via IA e homologado
```

Artefato	Quem escreve	IA pode alterar?
`README.md`	Humano	Somente com autorização
`CONTEXT_RULES.md`	Humano	Não
`specs/task_analyzer_spec.md`	Humano	Não
`tests/test_harness.py`	Humano	Não (P02)
`src/task_analyzer.py`	IA + revisão humana	Sim, dentro das regras
`requirements.txt`	Humano	Não (P01)
`.gitignore`	Humano	Não

4.1 Plano de homologação humana

Passo	Verificação	Critério
1. Testes	`python -m pytest -v --cov=src --cov-report=term-missing`	100% aprovados; cobertura ≥ 90%.
2. Conformidade	Código × contrato (1.3) e regras D01–D12 / P01–P12.	Assinaturas idênticas a 1.3.5.
3. Qualidade	Legibilidade, docstrings, erros, tamanho das funções, logs.	Nenhum `print`, `except Exception` ou import externo.
4. Testes manuais	Casos extras no terminal (volume, fusos variados).	Resultados coerentes com as RN; anotados no PR.
5. Aprovação	Merge do PR na `main` após os passos 1–4.	Commit identificado como assistido por IA.

Rejeição: teste falhando, violação de proibição ou comportamento não especificado implicam rejeição do código e nova solicitação à IA com o motivo explícito.

4.2 Versionamento

- Branches: `main` (somente código homologado) e `feature/<descricao>`: `feature/sdd-specification`, `feature/test-harness`, `feature/task-analyzer-impl`.
- Conventional Commits: `feat:`, `test:`, `docs:`, `fix:`, `refactor:`, `chore:`.
- Commits com código gerado por IA incluem o rodapé `AI-Assisted: sim (revisado por Pedro Vargas dos Santos e Silva)`.
- Pull Requests com o checklist de homologação (4.1) para integrar na `main`.
- Tags: `v0.1.0` (spec + testes) e `v1.0.0` (implementação homologada).